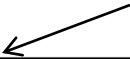


Graphs

Algorithms Related to Graph Connectivity

Depth-First Search (DFS)

A part of DFS:



Finding all nodes reachable from a particular node.

procedure explore(G, v)

Input: $G = (V, E)$ is a graph; $v \in V$

Output: $\text{visited}(u)$ is set to true for all nodes u reachable from v

$\text{visited}(v) = \text{true}$

$\text{previsit}(v) \longrightarrow$ Optional

for each edge $(v, u) \in E$:

 if not $\text{visited}(u)$: $\text{explore}(u) \longrightarrow$ Recursion only if
 $\text{visited}(u) = \text{false}$

$\text{postvisit}(v) \longrightarrow$ Optional

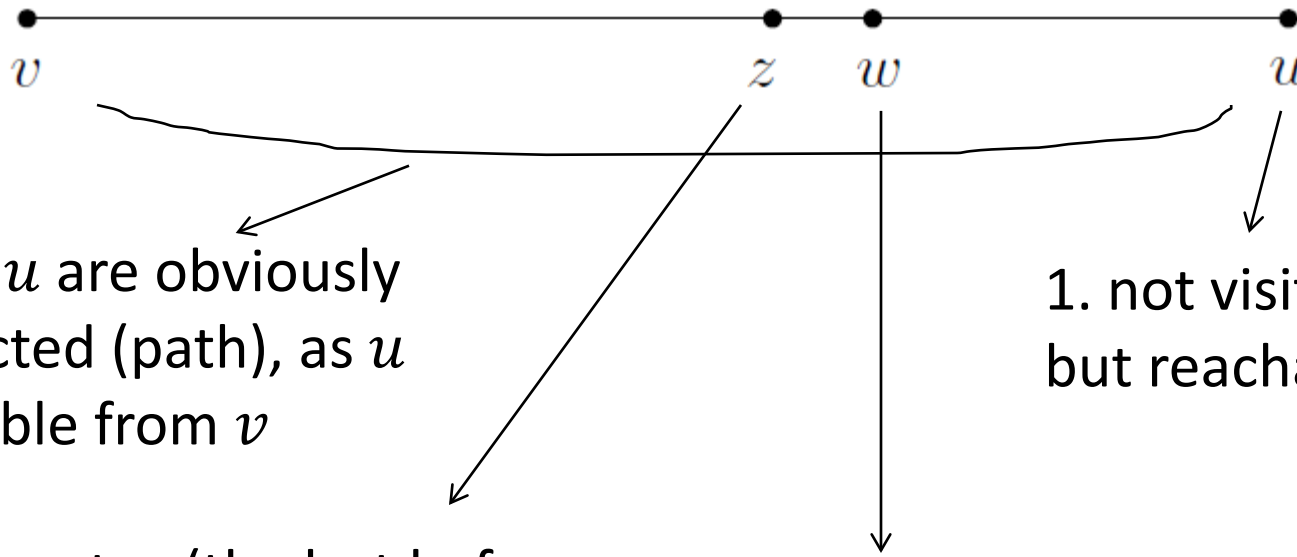


It moves only through connections and hence will not arrive at a node not reachable from v .

As this operates when only at previously not visited nodes, it is bound to halt at some point (and backtrack).

But will it find all reachable nodes from v before it halts?

Proof by contradiction. Say, no it won't.



2. v & u are obviously connected (path), as u reachable from v

1. not visited, but reachable

4. A vertex (the last before w) that the algo has visited in the path from v to u , such that it has an outgoing edge (z, w) [assured to have as the path exists].

3. The vertex 'nearest' to v in the path from v to u that has not been visited by the algo.

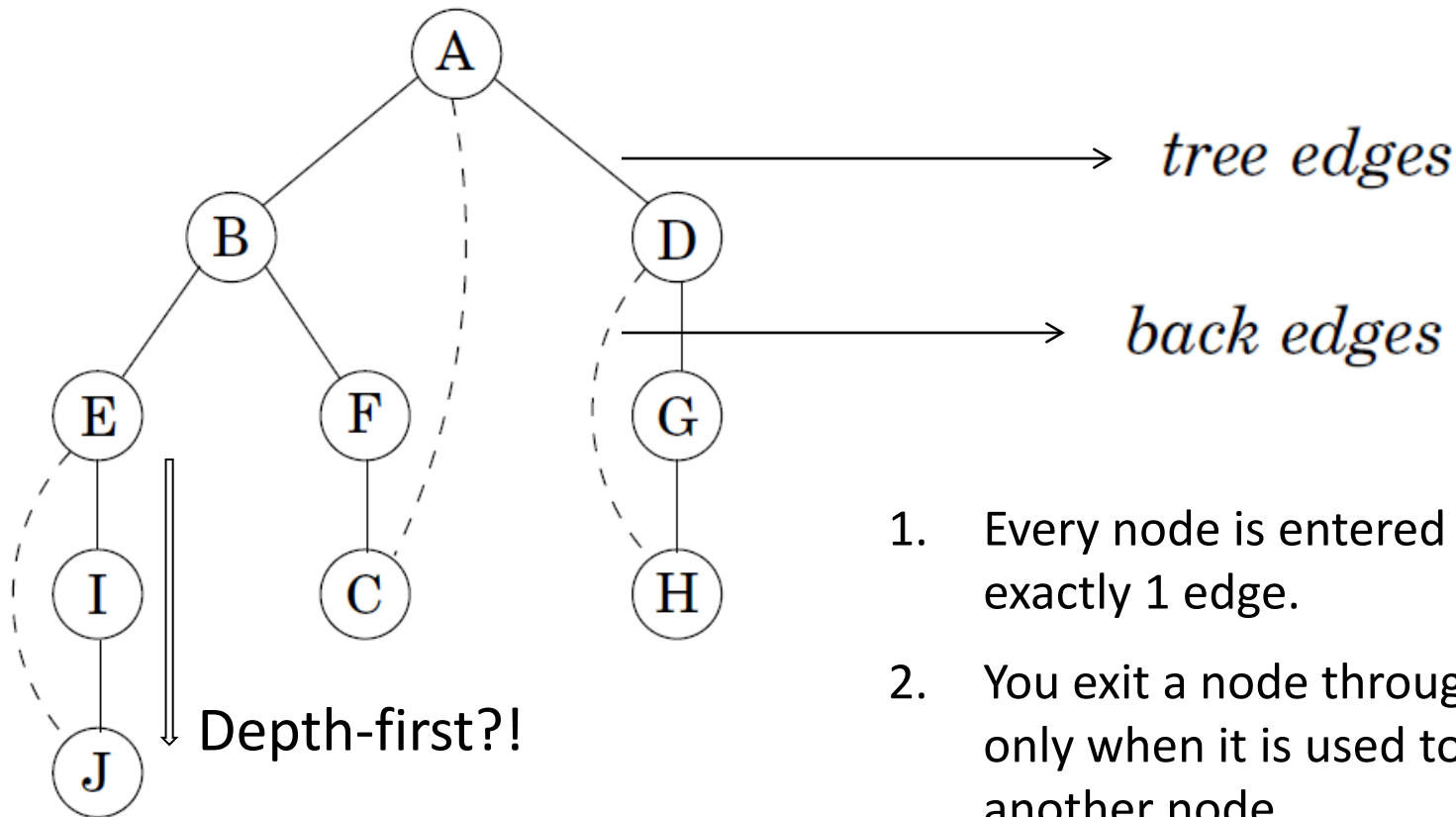
This is a contradiction! If this edge is there then w must have been visited!

Formally:

“for any $k \geq 0$, all nodes within k hops from v get visited.”

if all nodes k hops away are visited, then so are all nodes $k + 1$ hops away

explore(A)



1. Every node is entered through exactly 1 edge.
2. You exit a node through an edge, only when it is used to enter another node.

solid edges form a tree (a connected graph with no cycles)

Depth-first search.

procedure `dfs`(G)

for all $v \in V$: $O(n)$
 `visited`(v) = false

for all $v \in V$:
 if not `visited`(v): `explore`(v)

All this is going to do is to mark / visit all the vertices in the graph. So the entire graph will be traversed by visiting each node once.

What's the use? We will see.

During the exploration of a vertex, following is done:

1. Some fixed amount of work—marking the spot as visited, and the `pre/postvisit`.
2. A loop in which adjacent edges are scanned, to see if they lead somewhere new.

Total work done in Step 1: $O(n)$ $O(|V|)$

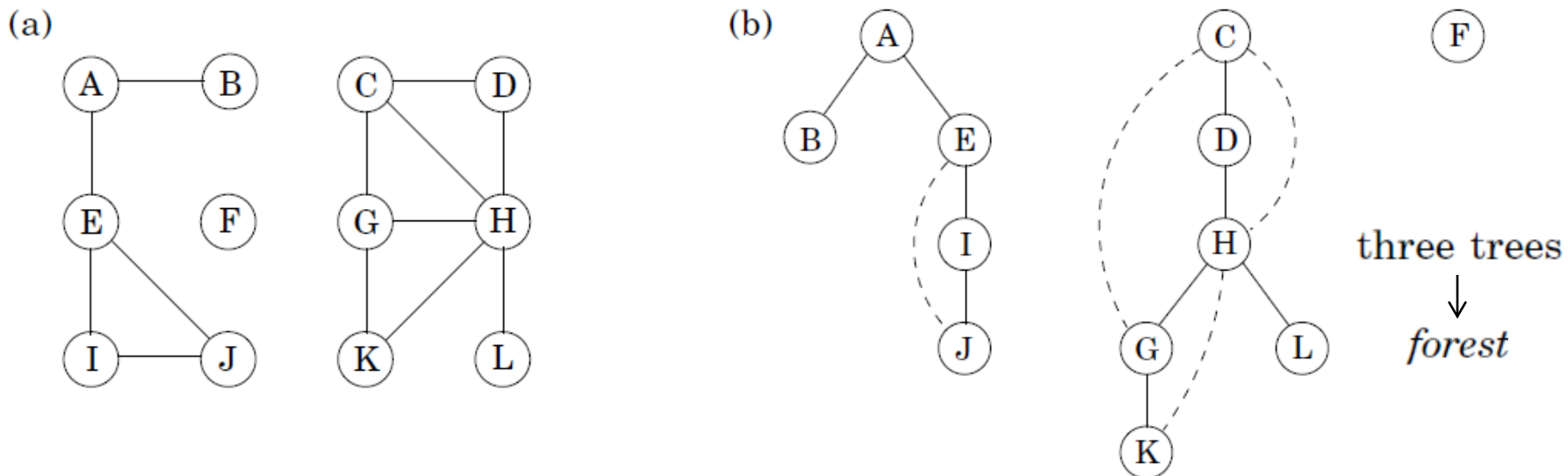
Each vertex $x \in V$ is visited for the fixed work once.

Total work done in Step 2: $O(m)$ $O(|E|)$

Each edge $(x, y) \in E$ is examined twice*, during explore (x) and explore (y).

So, DFS runtime: $O(n + m)$ $O(|V| + |E|)$

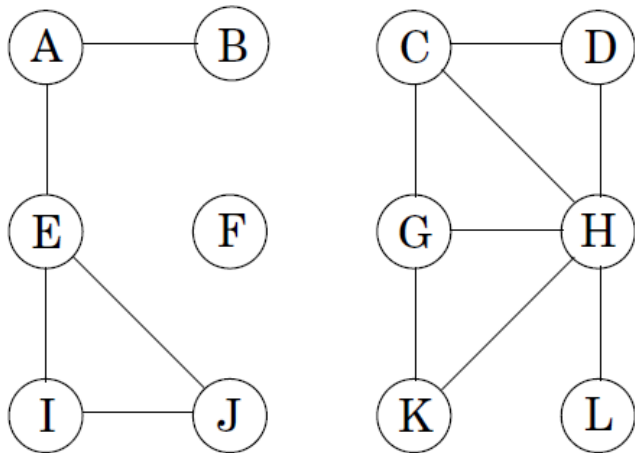
(a) A 12-node graph. (b) DFS search forest.



Directed graphs

Undirected graphs

An undirected graph is *connected* if there is a path between any pair of vertices.



Not connected.

Connected components (subgraphs)

$$\{A, B, E, I, J\}$$
$$\{C, D, G, H, K, L\}$$
$$\{F\}$$

Equivalence classes

Connection between any $u, v \in V$ in an undirected graph is an equivalence relation.

Task: Find all connected components of graph G

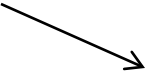
The Procedure:

```
for all  $v \in V$ :  
    visited( $v$ ) = false  
    ccnum[ $v$ ] = NULL
```

```
cc = 0
```

```
for all  $v \in V$ :  
    if not visited( $v$ ):  
        cc++  
        explore( $v$ )
```

Linear time!



```
    visited( $v$ ) = true
```

```
    procedure previsit( $v$ )  
        ccnum[ $v$ ] = cc
```

```
    for each edge  $(v, u) \in E$ :  
        if not visited( $u$ ):    explore( $u$ )
```

Procedure to store first discovery and final departure of / from a node:

```
for all  $v \in V$ :
    visited( $v$ ) = false
    pre [ $v$ ] = NULL
    post [ $v$ ] = NULL

clock = 1
for all  $v \in V$ :
    if not visited( $v$ ):
        explore( $v$ )
```

```
visited( $v$ ) = true
procedure previsit( $v$ )
```

```
pre [ $v$ ] = clock
```

```
clock = clock + 1
```

Stack: push,
store

```
for each edge  $(v, u) \in E$ :
```

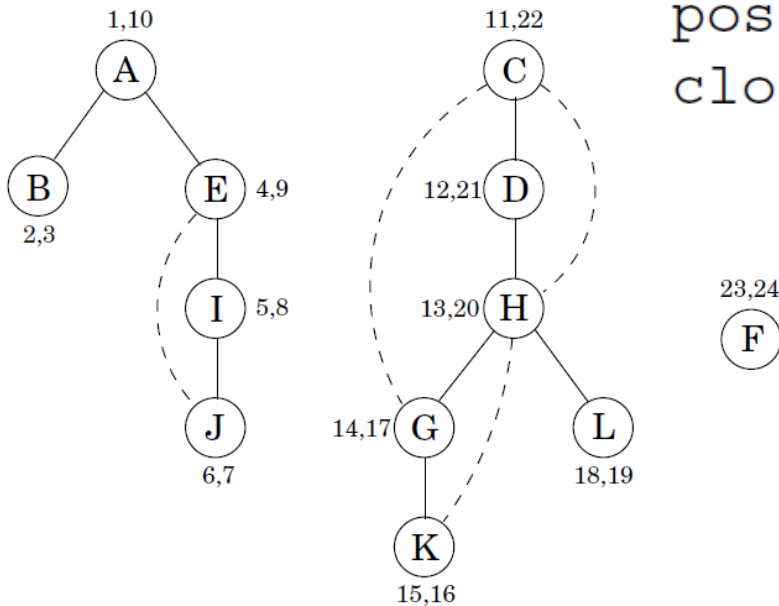
```
    if not visited( $u$ ): explore( $u$ )
```

```
procedure postvisit( $v$ )
```

```
post [ $v$ ] = clock
```

```
clock = clock + 1
```

Stack: pop,
check, reset



*For any nodes u and v , the two intervals $[pre(u), post(u)]$ and $[pre(v), post(v)]$ are either disjoint or one is contained within the other.

Types of interval:

- No intersection (other than subsets) of intervals
- Possible: Intervals subsets of other intervals
- Possible: Disjoint Intervals
- Number of connected components = Number of intervals that are not subsets of other sets (except universe)
 - Elements within such a set are the connected components!
(use Stack DS, emptied stack indicates a connected component)

Linear time!

Directed graphs

What happens when we run the above algo in a directed graph??

